Autoencoder for Credit Card Fraud Detection

Dana Ibrahim
Department of Computer Science
University of Texas at Dallas
Richardson, TX, USA
dyi230000@utdallas.edu

Sumaiya Lana
Department of Computer Science
University of Texas at Dallas
Richardson, TX, USA
ssl240001@utdallas.edu

Sabiba Rudana
Department of Computer Science
University of Texas at Dallas
Richardson, TX, USA
srr220008@utdallas.edu

***Abstract*—Credit card fraud detection is a critical need for financial institutions as they may cause billions of dollars in financial losses annually. This paper presents an autoencoder based anomaly detection built and trained exclusively on normal credit card transactions to learn a comprehensive but compressed representation of legitimate behavior. Fraudulent transactions are identified by primarily measuring reconstruction errors. This happens as anomalous transactions deviate significantly from normal patterns and are reconstructed poorly by the model. The model achieves a ROC-AUC score of 0.9553 which demonstrates a strong discriminative ability despite the class imbalance present in the dataset.**

Keywords—autoencoder, anomaly detection, credit card fraud, reconstruction error, neural networks

I. Introduction

Financial fraud is a rapidly growing challenge that affects both individuals and institutions. According to recent studies, credit card fraud alone results in tens of billions of dollars in losses every year. Although traditional rule-based fraud detection systems are limited because they require manual updates and can potentially fail to adapt to new ad evolving fraud patterns, machine learning offers a promising alternative by allowing systems to learn directly from data without the need for explicit rules.

Among various machine learning techniques, autoencoders have materialized as a particularly effective approach for anomaly detection in scenarios where labeled fraud data is either scarce or heavily imbalanced. An autoencoder is a type of neural network that learns to compress input data into a lower dimensional representation and then reconstruct it back to its original form. When a model is trained only on normal transactions, it becomes specialized in reconstructing normal behavior. Since fraudulent transactions are structurally different from normal ones, they result in high reconstruction error and can therefore be flagged as anomalies.

In this paper, we implement an autoencoder without relying on any machine learning frameworks. We apply our model to a real-world credit card transaction dataset and evaluate its ability to detect fraudulent transactions using precision, recall, F1-score, and ROC-AUC. We also analyze the effect of varying the anomaly detection threshold on model performance.

II. Background Work

Autoencoders have been widely studied for anomaly detection across a variety of domains. Screyer et al. [1] proposed the use of deep encoder networks to identify anomalous journal entries in large-scale accounting data. The authors demonstrated that autoencoders can effectively identify deviations from normal financial behavior without relying on labeled anomaly data. Their work shed light on the practical value of reconstruction-based anomaly detection in real-world financial settings. What made their approach particularly interesting is that the model did not require examples of fraud to be shown to learn how to detect it. Contrarily, it was trained only on normal transactions and learned to flag anything it could not reconstruct up to the standards set forth. They tested this on real enterprise accounting data containing millions of records and showed that reconstruction error was a reliable signal for catching unusual entries. This directly inspired our approach since our model upholds the same principle of training only on normal data and using reconstruction error to identify fraud.

DiOng et al. [2] introduced an enhanced autoencoder method combined with LightGBM for credit card fraud detection that shows that autoencoders are effective for feature reconstruction on highly imbalanced financial datasets and that an amalgamation of reconstruction error and traditional classifiers can potentially improve the detection performance of a model. Their findings suggest that the reconstruction error produced by autoencoders carries meaningful information that can significantly boost downstream classified performance. Having said that, what made their work truly stand out was that they did not treat the autoencoder as the final detector, rather, they used it as a feature extraction tool that feeds useful information into a separate classifier. They also brought forth something crucial that many papers have the propensity to overlook. Standard accuracy is not a good measure when fraud cases make up led than one percent of the data, which is why they evaluated their model using additional metrics like Matthews Correlation Coefficient and Balanced Classification Rate to get a more authentic picture of the performance. Ultimately, their results showed strong improvements over baseline methods across two separate financial datasets which gave us confidence that reconstruction error is a meaningful signal worth building on in our work.

Onyeama et al. [3] compared autoencoders against Principal Component Analysis (PCA) for fraud detection on financial data. The author concluded that autoencoders offer stronger performance in dimensionality reduction and produce more reliable anomaly detection results on complex datasets.

XXX-X-XXXX-XXXX-X/XX/$XX.00 ©20XX IEEE

This comparison reinforced the idea that autoencoders are particularly well suited for high-dimensional financial data where linear methods fall inadequate. The reason behind this comes down to how much each method works. PCA can only capture linear relationships in the data while autoencoders are capable of learning complex nonlinear patterns that PCA simply cannot represent. In the context of credit card fraud this matters a lot because fraudulent behavior is often subtle and deeply embedded in combinations of features only a nonlinear model can potentially pick up on. The authors also pointed out that fraudsters do not follow predictable patterns and constantly adapt their behavior over time. This makes it even more important to use a model that can capture complex and shifting relationships in the data rather than one that simply assumes everything follows a straight line. Another important takeaway from their work was the hardship of obtaining reliable financial datasets for research purposes. Real transaction data is sensitive and are rarely made publicly available which limits how broadly results can be tested and compared across different studies. This is a limitation our project shares as well since we relied on a publicly available dataset that may not fully capture the complexity of real world fraud patterns.

Augustin et al. [4] explored radial autoencoders specifically designed to handle class imbalance in anomaly detection tasks. The author highlighted that standard autoencoders trained only on majority class data naturally align with one class classification principles which makes them suited for fraud detection. Their work emphasized the importance of threshold selection in determining the tradeoff between precision and recall which is a challenge focal to fraud detection systems currently in practice. This was one of the most relevant findings for our own work. There is no single perfect threshold that works for every situation. Raising the threshold means fewer false alarms but more missed fraud cases. Lowering it means catching more fraud but also flagging more legitimate transactions. The right choice, however, depends on what the application can tolerate. This is exactly why we analyzed multiple threshold values in our experiments to understand how each one affects the balance between precision and recall rather than just picking one and reporting a single result.

Our work builds on these findings by implementing a standard feedforward autoencoder and evaluating its effectiveness. By coding all components manually, our aim is to demonstrate a transparent and interpretable approach to fraud detection that is accessible and reproducible by others.

III. Theoretical and Conceptual Study

A. Autoencoder Overview

We have built an autoencoder for the Fraud detection case. The biggest problem with fraud detection is that it is very rare, leading to very little data to work with. The autoencoder helps with this problem, because it is a neural network that learns what normal transactions look like. This helps because essentially the autoencoder is trying to detect abnormalities in patterns from normal transactions to detect fraudulent transactions.

An autoencoder is a type of neural network that learns to compress data and rebuild it. The two main components of the autoencoder are the encoder, which takes the input and compresses the data by preserving the most significant features, and the decoder takes the compressed version and tries to rebuild it back to the original version as closely as possible. The filtering/compression process is the most important, since the identification of which features are significant, and which need to be filtered out must be well chosen.

How it works for fraud detection is that when the encoder has compressed the data and it is time for the decoder to rebuild the data, it will have a poor reconstruction. Since there will be inconsistencies with the pattern, the autoencoder has learned from normal transactions. The poor reconstruction is going to result in a high error, leading to being flagged as fraud.

B. How the Network is Structured

For this project, we use a transaction with 30 features for the input to the network. The encoder passes the input through two layers; it shrinks the 30 neurons to 16 neurons, then to 8 neurons. 8 neurons are the middle point, bottleneck, also referred to as the latent space. The latent space is the compressed version of the initial transaction. Then the decoder will take latent space and work its way up, 8 --> 16, then finally back to 30 neurons to rebuild the original input that was given.

Each layer will multiply the input by a weight matrix, add a bias, and apply an activation function. The activation function we use is the ReLU, Rectified Linear Unit. It is defined as ReLU(x) = max(0,x), it keeps the positive values as is and will turn all negative values to 0. ReLU is simple and helps the network train faster without running into the vanishing gradient problem, which functions like sigmoid functions can run into. The last layer of the decoder does not use ReLU because the reconstructed values need to be able to go negative after standardization.

$$z_{j,i}^{[l]} = \sum_{k} w_{j,k}^{[l]} \cdot a_{k,i}^{[l-1]} + b_j^{[l]}$$

Fig. 1. Mathematical representation of the linear transformation in the l-th layer of a neural network [5].

C. Training the model

The model is being trained by comparing the reconstructed input from the decoder to the original input. The comparison is measured through the MSE, Mean Squared Error shown in Fig. 2, which will average the squared differences between each feature.

$$\mathrm{MSE} = \frac{1}{n} \sum_{i=1}^{n} \left(Y_i - \hat{Y}_i \right)^2$$

Fig. 2. Equation for Mean Squared Error (MSE) [7].

The reason the difference is squared is that the bigger mistakes get penalized more heavily, which will be useful for fraud detection. The model will then use backpropagation to adjust its weight based on this error; this will work backwards from the decoder and encoder. To be able to keep the training stable, gradients are clipped to stay within +- 1.0. The model will train in batches of 256 transactions at a time, with a learning rate of 0.003, and stop early if the loss stops improving over 20 epochs.

D. *Detecting Fraud using Reconstruction Error*

Once the model is trained, detecting fraud is done by looking at how badly it reconstructs each transaction. A threshold is then set based on the reconstruction error from normal transactions. Any transaction that has an error above the threshold will be flagged as fraud. In this project, we tested the threshold at different percentile levels (90th – 99th) percentile. Within this, the 97th percentile gave the best balance between catching fraud and not flagging many normal transactions as suspicious. The approach works without needing any labeled fraud examples during the training process.

IV. Results and Analysis

A. *Training Performance*

The autoencoder was trained using only normal (non-fraudulent) credit card transactions so that it could learn what "normal" behavior looks like. During training, the training loss kept decreasing, which shows that the model was learning the patterns of a normal transaction and rebuilding the input data. At the beginning, the loss was approximately 1.01, and it slowly went down to around 0.90. At 255 epochs, the training stopped early because the model stopped improving and the average loss value even started going up instead. The training process was stable, and the loss trend shows that the model learned a good generalization of normal transactions and did not overfit.

B. *Loss Curve Analysis*

As shown in Fig. 3, the training loss curve decreases over time, showing us the model stably converges and is not overfitting. There aren't any jumps or spikes in the curve, which usually means the training process was working properly. After a while, the curve starts flattening out, meaning the model has reached its best performance and is unlikely to learn more from adding more epochs. Early stopping was triggered in at the end, which helped avoid wasting time training when the model wasn't improving anymore.

C. *Threshold Tuning Analysis*

Since this is an anomaly detection model, choosing the right threshold is very important. Instead of picking one value right away, we tested different percentile thresholds using reconstruction error. The results in Table I show that there is a noticeable tradeoff between precision and recall. Lower thresholds (such as 90-93%) had a higher recall, meaning they caught more fraud cases, but they also produce a lot of false positives. On the other hand, higher thresholds reduce the number of false positives but then they miss more fraud transaction cases. Therefore, a middle range of around 95-97% made the most sense for final testing.

In the first experiment, a threshold of 95% was used. The result was a high recall of 0.8760, however the number of false positives was still pretty high with 13,971 normal transactions incorrectly classified as fraud, with 27,0344 true negatives. To improve this, the threshold was increased to 97% in the second experiment. This resulted in an increase in the precision from 0.0299 to 0.0476 while recall only dropped slightly from 0.8760 to 0.8598. Most importantly, this threshold lowered the number of false positives from 13,971 to 8,463. In the end, the decision was to use the 97% threshold because it gave a better balance between detecting fraud and reducing false positives.

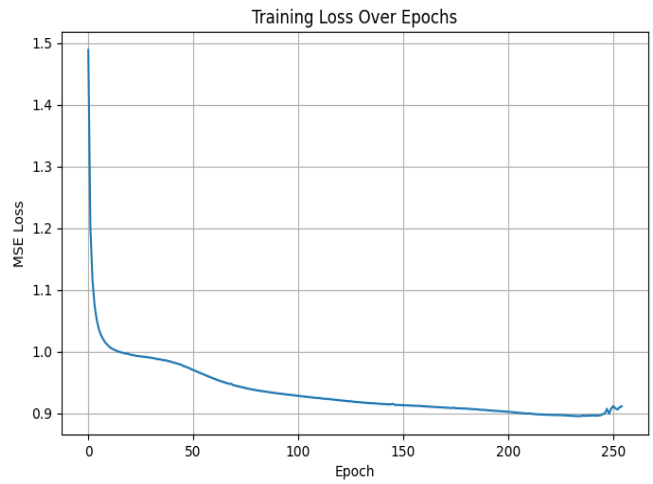

Fig. 3. Training loss curve of the autoencoder over epochs showing convergence behavior (generated from code).

TABLE I. Threshold Tuning Results

Percentile	*Threshold*	*Precision*	*Recall*	*F1 Score*
90	1.3067	0.0155	0.9065	0.0305
93	1.5593	0.0217	0.8943	0.0425
95	1.8636	0.0299	0.8760	0.0579
97	2.5636	0.0476	0.8598	0.0902
99	6.3316	0.0992	0.6423	0.1718

D. *Final Model Performance*

The final model was evaluated on the full dataset using the selected threshold of 97% (threshold = 2.5636). The autoencoder architecture included three layers in both the encoder and decoder, with neuron sizes of (30, 16, 8, 16, 30). The model was trained using Mean Squared Error (MSE) as the

loss function, with a learning rate of 0.003 and a batch size of 256 and a maximum of 300 epochs. The train-test split used was 80:20 and early stopping was used during the training.

The performance results of the final model are summarized in Table II. The model had a precision of 0.0476, a recall of 0.8598, and a F1 score of 0.0902. The high recall means the model is effective at identifying fraudulent transactions, while the relatively low precision makes sense with the presence of false positives, which is common in imbalanced datasets where fraudulent transactions only make up a small fraction of the total data. In these cases, models tend to produce more false positives in order to ensure that fraudulent cases are not missed. In addition, the model achieved a ROC-AUC score of 0.9553, which is relatively high and means that the model is able to distinguish between normal and fraud transactions based on the reconstruction error.

E. Confusion Matrix Analysis

The confusion matrix for the final model is shown in Table III. The confusion matrix is helpful for viewing how the model performs across different prediction categories. Out of the total transactions, 275,852 normal transactions were correctly classified as non-fraudulent (true negatives), while 8,463 normal transactions were incorrectly classified as fraud (false positives). For the fraudulent transactions, the model correctly identified 423 cases (true positives) and incorrectly classified 69 cases (false negatives).

These results show that the model prioritizes correctly detecting fraudulent transactions, as seen by the relatively low number of false negatives. This is especially important in fraud detection, where missing a fraudulent transaction can have serious effects and consequence. In real world financial systems, false negatives are usually more costly than false positives because undetected fraud can lead direct financial loss.

However, the number of false positives is still relatively high, which could lead to inconvenience to users, such as unnecessary transaction blocks or verification steps. Because of this, the model would still possibly need to be combined with additional filtering or a secondary verification system. Overall, these results highlight the trade-off that exists between precision and recall discussed in the threshold tuning section, as well as the importance of balancing machine learning model performance with practical system design considerations.

TABLE II. FINAL MODEL EVALUATION RESULTS

Metric	***Value***
Precision	0.0476
Recall	0.8598
F1 Score	0.0902
ROC-AUC	0.9553

TABLE III. FINAL CONFUSION MATRIX

	Predicted Normal	***Predicted Fraud***
Actual Normal	275,852 (TN)	8,463 (FP)
Actual Fraud	69 (FN)	423 (TP)

V. CONCLUSION AND FUTURE WORK

In this project, we have built an encoder for credit card fraud detection without relying on any machine learning frameworks. The biggest challenge with fraud detection is the lack of labeled fraud data, and the autoencoder helps with this problem. The autoencoder addresses that by utilizing the normal transactions learning from them and then identifying pattern inconsistencies to flag fraud based on the reconstruction error.

The model performed well, it has achieved a ROC-AUC score of 0.9533, showing that it is able to separate normal transactions from fraudulent ones effectively despite the heavy class imbalance in the dataset. The 97th percentile threshold gave the best balance between catching fraud and keeping false alarms at a low level. One limitation of this current approach is that the threshold is chosen manually by testing different percentiles, which may not be a suitable threshold for all datasets.

The training was stable, with the loss steadily decreasing and early stopping preventing any unnecessary training once the model stopped improving. Overall, the autoencoder proved to be an effective approach for anomaly-based fraud detection, especially in situations where labeled fraud examples are scarce or hard to come by.

There are a few things we believe can be improved. One thing we would like to do is test different architectures, such as adding more layers or changing the size of the bottleneck, to see if the reconstruction quality improves. Additionally, we would like to try coming to the autoencoder with a traditional classifier, where the reconstruction error is used as a feature, since it has been shown that this can boost detection performance.

Finally, testing the model on newer or different fraud datasets would help confirm whether it generalizes well beyond the dataset used in this project.

REFERENCES

[1] M. Schreyer, T. Sattarov, D. Borth, A. Dengel, and B. Reimer, "Detection of Anomalies in Large Scale Accounting Data using Deep Autoencoder Networks," *arXiv:1709.05254 [cs]*, Aug. 2018, Available: https://arxiv.org/abs/1709.05254

[2] L. Ding, L. Liu, Y. Wang, P. Shi, and J. Yu, "An AutoEncoder enhanced light gradient boosting machine method for credit card fraud detection," *PeerJ Computer Science*, vol. 10, p. e2323, Oct. 2024, doi: https://doi.org/10.7717/peerj-cs.2323.

[3] J. Onyeama, "Credit Card Fraud Detection in the Nigerian Financial Sector: A Comparison of Unsupervised TensorFlow-Based Anomaly Detection Techniques, Autoencoders and PCA Algorithm," *arXiv.org*, 2024. https://arxiv.org/abs/2407.08758

[4] M.-C. Augustin, V. Bonvin, R. Houssou, E. Rappos, and S. Robert-Nicoud, "Radial Autoencoders for Enhanced Anomaly Detection," *arXiv.org*, 2022. https://arxiv.org/abs/2203.15884 (accessed May 05, 2026).

[5] L. Samani, "From Theory to Practice: Building a Deep Feedforward Neural Network with Back Propagation in Python," *Medium*, Apr. 11, 2024: https://medium.com/@samaniloqman91/from-theory-to-practice-building-a-deep-feedforward-neural-network-with-back-propagation-in-python-9edc39164a68 (accessed May 05, 2026).

[6] GeeksforGeeks, "ReLU Activation Function in Deep Learning," *GeeksforGeeks*, Sep. 30, 2024. https://www.geeksforgeeks.org/deep-learning/relu-activation-function-in-deep-learning/

[7] Wikipedia Contributors, "Mean squared error," *Wikipedia*, Mar. 30, 2019. https://en.wikipedia.org/wiki/Mean_squared_error

[8] "Credit Card Fraud Detection," Kaggle. [Online]. Available: https://www.kaggle.com/datasets/mlg-ulb/creditcardfraud